

# Robotic Navigation and Exploration – HW3–1

簡誌加 N26140692

# 深度強化學習：Proximal Policy Optimization (PPO) 實作

在這部分的作業中，我們實作了 PPO 演算法來進行機器人的連續動作空間控制。主要包含了以下幾個核心實作：

1. **Actor–Critic 網路架構**：建構 Policy Network 與 Value Network。
2. **環境互動與資料收集**：在 `EnvRunner` 中與環境互動並蒐集訓練用的 Batch Data。
3. **PPO 損失函數運算**：計算具備 Clip 機制的 Policy Gradient Loss 來穩健地更新策略。

## TODO 1 & 2: Policy & Value Network 架構

我們在 `model.py` 中實作了 Actor-Critic 的雙網路架構，使用多層的全連接層 (MLP)，並搭配 PPO 處理連續動作空間所需的常態分佈 (`DiagGaussian`)。

```
# Policy Network (Actor)
self.main = nn.Sequential(
    init_(nn.Linear(s_dim, 64)),
    nn.ReLU(),
    init_(nn.Linear(64, 64)),
    nn.ReLU()
)
self.dist = DiagGaussian(64, a_dim, std)

# Value Network (Critic)
self.main = nn.Sequential(
    init_(nn.Linear(s_dim, 64)), nn.ReLU(),
    init_(nn.Linear(64, 64)), nn.ReLU(),
    init_(nn.Linear(64, 1))
)
```

## TODO 3: 環境資料收集 (EnvRunner)

利用模型在環境中實際前進探索。使用 `torch.no_grad()` 避免計算不必要的梯度以提升效能，產出動作、預估的 Value 與動作的 Log Probability。

```
with torch.no_grad():
    states_tensor = torch.from_numpy(self.states).float().to(self.device)
    actions_tensor, a_logps_tensor = policy_net(states_tensor)
    values_tensor = value_net(states_tensor)

actions = actions_tensor.cpu().numpy()
a_logps = a_logps_tensor.cpu().numpy()
values = values_tensor.cpu().numpy()

# 將探索值寫入 Buffer，並在環境中往下走一步
self.mb_states[step, :] = self.states
self.mb_actions[step, :] = actions
self.states, rewards, self.dones, info = self.env.step(actions)
```

## TODO 4: PPO 策略梯度損失 (Policy Gradient Loss)

PPO 的核心精神是透過 Clip Mechanism，將新舊策略比值 *ratio* 限制在  $[1 - \epsilon, 1 + \epsilon]$  的範圍內，避免單次更新幅度過大導致模型崩潰。

```
# 計算新舊策略的機率比值 ratio = exp(log_prob_new - log_prob_old)
ratio = torch.exp(sample_a_logps - sample_old_a_logps)

# Surrogate objectives
surr1 = ratio * sample_advs
surr2 = torch.clamp(ratio, 1.0 - self.clip_val, 1.0 + self.clip_val) * sample_advs

# 取兩者之中較悲觀(較小)的值，並加上負號轉為最小化(Minimize)問題
pg_loss = -torch.min(surr1, surr2).mean()
```

透過這種方式，演算法既能保有 Actor-Critic 的良好採樣效率，又確保了訓練過程的穩定性！